Computação Embarcada

Aula 01: Fundamentos, Arquiteturas e Aplicações no Mundo Real

Apresentação e Dinâmica

Diego Fernando Maia

📧 dmaia@up.edu.br

🎓 Formação

Mestre em Telecomunicações e Informática

UTFPR – Universidade Tecnológica Federal do Paraná • 2025

Tecnologia em Sistemas de Telecomunicações

UTFPR – Universidade Tecnológica Federal do Paraná • 2015

💼 Experiência Profissional (14+ anos)

- Volvo do Brasil
- Uni.Cloud
- Electrolux do Brasil
- MOB Telecom
- IGN Telecomunicações
- Telefonica/Vivo
- GVT

Organização da Disciplina

Ementa, Metodologia, Avaliações e Componentes

Objetivos & Ementa

O que vamos aprender?

- Compreender a **arquitetura de sistemas microcontrolados**, suas partes internas e conexão com elementos externos.
- Configuração de periféricos (GPIO, Memória) e unidades funcionais como Timers, Interrupções e Interfaces.
- O foco principal é **sair do nível teórico e criar protótipos funcionais** que resolvam problemas reais em alto e médio nível.

Tópicos Abordados

- Introdução a plataformas microcontroladas.
- Arquitetura, organização e características.
- Configuração e programação em C/C++ focada em hardware de recursos limitados.
- Projeto prático de Sistemas Microcontrolados para Internet das Coisas (IoT).

Condução das Aulas

Exposição Teórica

Apresentação e análise de arquiteturas e conceitos com utilização de multimídia. Entendimento profundo do funcionamento em baixa complexidade e aplicações em alto nível.

Exercícios de Fixação

Realização de exercícios teóricos em sala. O desenvolvimento deve sempre ser apresentado de forma legível, organizada e dentro da data estipulada.

Laboratório (Hands-On)

Atividades práticas na bancada. Montagem eletrônica e desenvolvimento de código embarcado na plataforma Arduino para resolver problemas aplicados.

Carga Horária e Avaliações

Estrutura dos Encontros

Disciplina de 60h: 3 horas-aula consecutivas por semana.

Horários:

- 19:00 – 20:15 (Primeira parte)
- 20:15 – 20:35 (Intervalo)
- 20:35 – 21:50 (Segunda parte)

Sistema de Notas (A1 + A2)

- Prova teórica individual: **50% da nota**
- Trabalhos e relatórios práticos: **50% da nota**

Mínimo para aprovação: NF ≥ 6,0 e 75% de frequência.

** A Avaliação Final Substitutiva (AF) substituirá a menor nota caso a NF seja inferior a 6,0.*

Regras de Convivência

Prazos Implacáveis

Trabalhos entregues em até 3 dias após o prazo original terão **desconto de 50%** na nota máxima. Após esse período extra, a entrega não será mais aceita.

Integridade Acadêmica

"No pain, No gain." O aprendizado exige esforço. Trabalhos que apresentarem evidências de cópia (plágio de colegas ou internet) terão **nota zero** automática.

Ambiente Colaborativo

Uma sala de aula é uma orquestra. Brincadeiras e perguntas são bem-vindas no momento certo. Conversas paralelas no momento errado atrapalham o aprendizado de todos.

Material de Laboratório

Para o desenvolvimento das atividades práticas neste semestre, utilizaremos o seguinte ecossistema:

- **Plataforma:** Placa de desenvolvimento Arduino UNO (ou equivalente compatível).
- **Eletrônica Básica:** Jumpers, Protoboard, resistores de valores diversos, capacitores, LEDs coloridos e um cartão de memória MicroSD.
- **Software:** Utilizaremos a Arduino IDE (oficial) para compilação e upload dos códigos C++ (Sketches).
- **AVA:** Todo o material (códigos base, relatórios padronizados e gabaritos) estará estruturado no portal da disciplina.

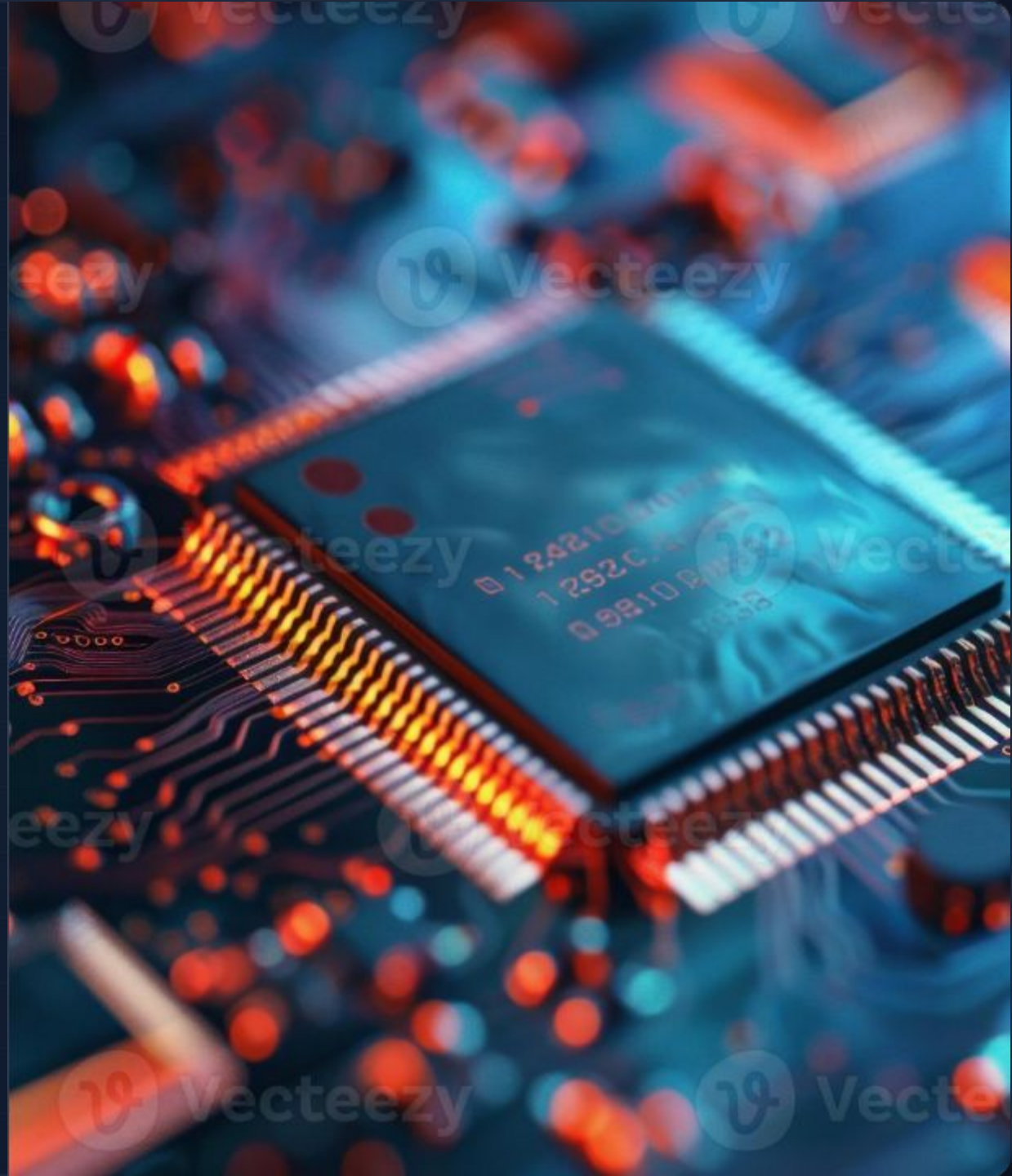
O que realmente define um sistema embarcado?

Computadores Dedicados

Um sistema embarcado é um sistema computacional projetado para realizar uma **tarefa específica**, frequentemente com restrições severas de processamento em tempo real.

Diferente de um PC que pode rodar desde jogos até planilhas simultaneamente, o dispositivo embarcado fica oculto, "embutido" dentro de um equipamento maior (um carro, um eletrodoméstico, um maquinário industrial).

O grande diferencial: Ele interage continuamente com o ambiente físico através de *Sensores* (entradas) e *Atuadores* (saídas).



Microprocessador vs Microcontrolador

Microprocessador (MPU)

É o cérebro que foca exclusivamente em **poder de processamento** matemático e lógico. Pense nos processadores Intel/AMD do seu PC.

- **Dependência:** Requer chips externos na placa-mãe para memória (RAM), armazenamento (SSD) e interfaces I/O.
- **Foco:** Altíssimo desempenho, GHz de velocidade, sistemas operacionais complexos (Windows, Linux, macOS).

Microcontrolador (MCU)

É um **computador inteiro em um único chip** (System on a Chip - SoC). Desenhado para controle autossuficiente e baixo custo.

- **Integração:** Traz a CPU, a Memória Flash (ROM), a RAM e os periféricos (Timers, ADC) fundidos no mesmo silício.
- **Foco:** Baixo consumo de energia (mW), custo na casa dos centavos, operação contínua e previsível.

As Arquiteturas de Processamento

Von Neumann vs Harvard

Von Neumann: Instruções do programa e dados dividem a mesma memória e o mesmo barramento. Sofre com o gargalo de leitura sequencial.
Harvard: Possui memórias e barramentos separados para Dados e Instruções. Pode ler uma instrução e um dado simultaneamente, ideal para tempo real.

CISC vs RISC

CISC: Hardware focado em interpretar comandos complexos com múltiplas operações de memória. (Ex: x86 Intel).
RISC: Conjunto Reduzido de Instruções. Comandos simples e rápidos executados em 1 ciclo de clock. Padrão dominante em embarcados por eficiência de energia.

Anatomia do Chip

CPU & Clock

A CPU busca instruções na Memória, decodifica e executa. O sinal de *Clock* dita o ritmo constante (ciclos) dessa operação. Em embarcados trabalha na escala dos Megahertz (MHz).

Memórias do MCU

A **ROM/Flash** (Não-volátil) guarda o Firmware - não apaga ao remover a energia. A **RAM** (Volátil) armazena variáveis e dados temporários enquanto o programa roda.

Periféricos (I/O)

Como o sistema interage. Pinos GPIO (liga/desliga), ADCs (leitura de tensão de sensores analógicos), Timers e controladores seriais (UART, I2C, SPI) conectando o mundo externo.

Aplicações e Dados Reais

Como a computação embarcada está transformando a economia mundial

O Avanço da Internet das Coisas (IoT)

30+ Bi

Dispositivos Conectados Globalmente

Estima-se que ultrapassaremos a marca de 30 bilhões de dispositivos IoT ativos entre 2026 e 2027, superando largamente a quantidade de humanos e computadores tradicionais na Terra.

1.5 Tri

Dólares em Valor de Mercado

O impacto econômico projetado para o ecossistema IoT (Hardware, Software embarcado e Serviços). A base dessa revolução multimilionária são pequenos microcontroladores conectados.

Indústria 4.0 e Edge Computing

Na Quarta Revolução Industrial, as linhas de produção ganham "cérebros" autônomos. Braços robóticos e esteiras não são apenas atuadores; eles coletam e processam dados ativamente.

Em vez de enviar terabytes de dados para a nuvem, utilizamos o **Edge Computing**: microcontroladores filtram dados de vibração e temperatura *diretamente na máquina* em milissegundos.

Dados Reais: Estudos apontam que a manutenção preditiva com sensores embarcados reduz os custos operacionais em até **40%** e minimiza falhas não programadas pela metade.

Automação: Residencial e Corporativa

A automação baseada em SoCs transformou tanto o conforto das residências quanto as infraestruturas empresariais:

- **Smart Homes:** Termostatos e protocolos como Zigbee/Matter rodam em microcontroladores que operam com baterias moeda por anos, garantindo eficiência energética (redução de até 15% de consumo).
- **Logística Conectada:** Rastreadores fundem GPS e 5G em placas minúsculas para monitoramento global de frotas.

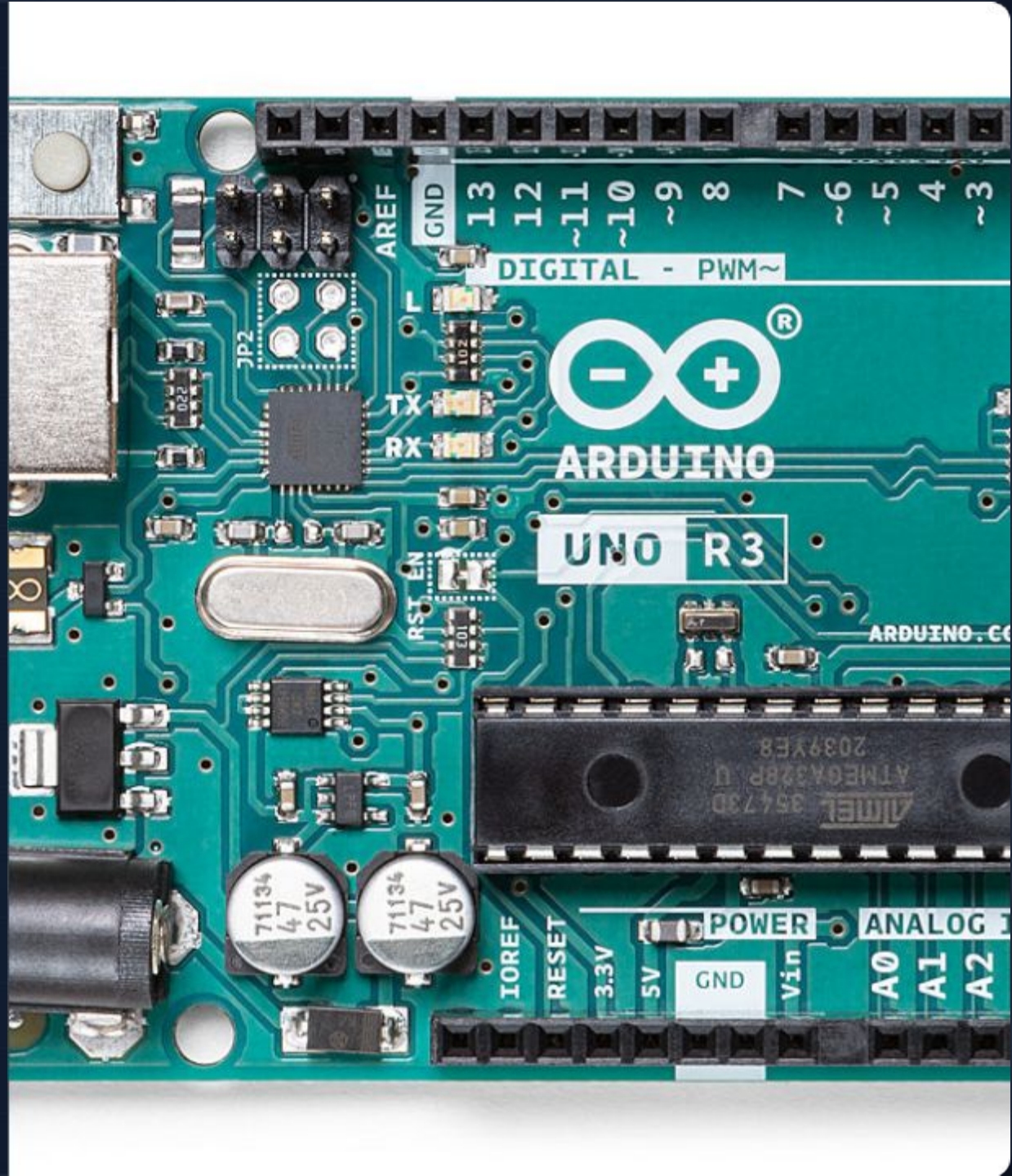
A Plataforma Arduino

O Ecossistema Open-Source

Nascido na Itália, o Arduino revolucionou a eletrônica ao oferecer uma plataforma de hardware e software livres que esconde a complexidade obscura dos datasheets.

A placa **Arduino UNO** é a porta de entrada para a engenharia computacional. Ela expõe os pinos do microcontrolador em conectores fáceis de usar (headers), permitindo prototipagem rápida na protoboard.

O poder verdadeiro vem das bibliotecas criadas pela vasta comunidade mundial, permitindo controlar sensores ultrassônicos e displays em poucas linhas.



Anatomia do Arduino UNO

O "Cérebro": ATmega328P

Um microcontrolador AVR de 8 bits operando a modestos 16 MHz. Possui 32KB de Memória Flash para o programa e apenas 2KB de RAM.
Essas severas restrições forçam o programador a escrever algoritmos eficientes, uma habilidade inestimável na nossa área da Computação.

Pinos e Conexões

Digitais (0 ao 13): Lidam com sinais binários lógicos (LOW = 0V, HIGH = 5V). Usados para ler botões ou acender LEDs.
Analógicos (A0 ao A5): Possuem Conversores Analógico-Digitais (ADC) de 10 bits. Permitem ler a variação contínua de sensores do mundo real.

A IDE e a Estrutura do Sketch

Programamos na **Arduino IDE**. A linguagem é baseada em C/C++, abstraindo o controle direto dos registradores de hardware com comandos facilitadores.

Todo código de Arduino (o *Sketch*) exige obrigatoriamente duas funções:

- void setup() { }
  Roda **apenas uma vez** ao ligar. Serve para configurar direções de pinos (INPUT/OUTPUT) e iniciar a Serial.
- void loop() { }
  O núcleo em tempo real. Um laço infinito que lê entradas e atualiza saídas repetidamente, até a placa ser desligada.

```
1349    d = String(e);
1350  SETSTRING:
1351    testTimeString = String(d) + String(":") + String(b) + String(":") + String(c);
1352    return;
1353  }
1354
1355
1356  void timeValueEdit() { // determines which section of the time screen is being ed
1357    // then calls timeValueEditIncDec() with the appropriate parameters
1358    int temp; // temporary variable for editing the year
1359    if ((cursorPosition == 0) || (cursorPosition == 1)) { // hour increase decrease
1360      hourSet = (byte)(timeValueEditIncDec(hourSet, 23)); // call function passing
1361    }
```

Colors for my VS Code, Code Clarity theme are shown above

```
1345    if (e < 100) {
1346      d = String("0") + String(e);
1347      goto SETSTRING;
1348    }
1349    d = String(e);
1350  SETSTRING:
1351    testTimeString = String(d) + String(":") + String(b) + String(":") + String(c);
1352    return;
1353  }
1354
1355
1356  void timeValueEdit() { // determines which section of the time screen is being ed
1357    // then calls timeValueEditIncDec() with the appropriate parameters
1358    int temp; // temporary variable for editing the year
```

O "Hello World" Físico: Blink

1. Preparação

O UNO possui um pequeno LED internamente ligado ao pino digital 13.

No bloco setup() declaramos a intenção:

pinMode(13, OUTPUT);

(Informamos à CPU que o pino 13 enviará tensão para fora).

2. Acionamento

Entramos no loop():

digitalWrite(13, HIGH);
(Aciona 5V = LED ON)

delay(1000);
(Pausa a operação por 1000 milissegundos)

3. Desligamento

Continuamos o laço:

digitalWrite(13, LOW);
(Corta a tensão = LED OFF)

delay(1000);
(Pausa 1s. O loop recomeça do topo).

Exercícios de Fixação Teórica

- **Questão 1:** Com base na arquitetura interna abordada hoje, cite e justifique quais são os 3 componentes cruciais que diferenciam fisicamente um Microcontrolador (MCU) de um Microprocessador (MPU).
- **Questão 2:** Analise as funções setup() e loop() do código Blink. O que ocorreria se a lógica de acendimento, espera e desligamento (as 4 linhas do loop) fossem movidas para dentro do bloco setup? Explique.
- **Questão 3:** Explique brevemente as vantagens da arquitetura de barramentos Harvard em comparação ao modelo Clássico de Von Neumann em sistemas operacionais que requerem execução precisa em tempo real.

Dúvidas?

Prof. MSc. Diego Fernando Maia | dmaia@up.edu.br